

Parte II

Arquitecturas Distribuidas

Capítulo 5

Semana 5: Estilos y Patrones de Arquitectura Distribuida

Fechas: 01 – 07 de junio de 2026

Semana: 5 de 18

Resultado de Aprendizaje: Aplica arquitecturas y patrones de diseño distribuido —microservicios, API REST, mensajería asíncrona, arquitectura en capas— para construir sistemas escalables, disponibles y seguros.

5.1 De los Monolitos a los Microservicios

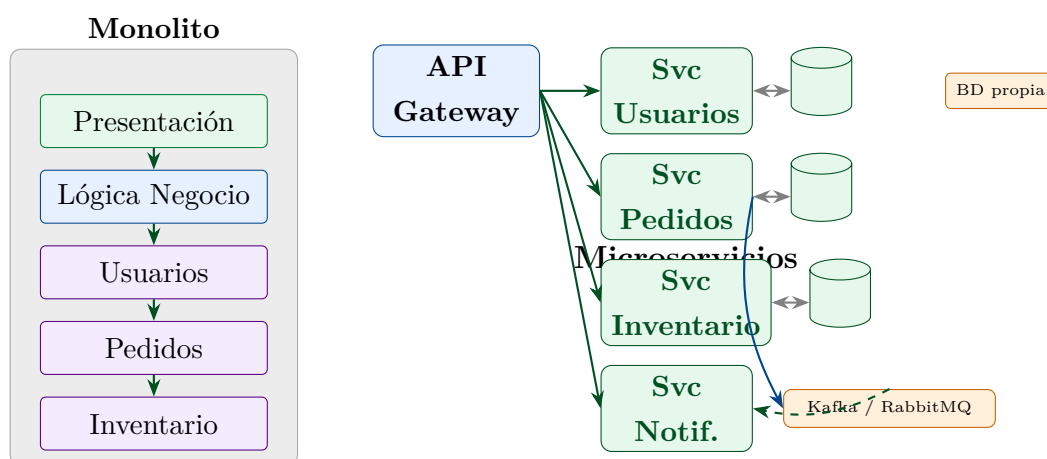


Figura 5.1: Transición de monolito a arquitectura de microservicios con API Gateway y bus de mensajes.

5.2 Microservicios con Spring Boot

Guía Práctica IDE – IntelliJ IDEA – Proyecto Spring Boot: svc-estudiantes

Paso 1: Usar **Spring Initializr** integrado en IntelliJ: *File* → *New* → *Project* → *Spring Initializr*. Configurar:

- Group: `ec.edu.uteq.distribuidas`
- Artifact: `svc-estudiantes`
- Java 21, Maven, Spring Boot 3.3.x
- Dependencias: **Spring Web**, **Spring Data JPA**, **H2 Database** (para dev), **Lombok**, **Spring Boot Actuator**, **Validation**

Paso 2: IntelliJ descarga las dependencias automáticamente. Verificar la estructura del proyecto en el panel de archivos.

Paso 3: Crear la estructura de paquetes:

```
src/main/java/ec/edu/uteq/distribuidas/  
    model/  
    repository/  
    service/  
    controller/  
    dto/  
    exception/
```

```
1 package ec.edu.uteq.distribuidas.model;  
2  
3 import jakarta.persistence.*;  
4 import jakarta.validation.constraints.*;  
5 import lombok.*;  
6 import java.time.LocalDateTime;  
7  
8 /**  
9  * Entidad JPA que representa un estudiante en el sistema  
10  * academico UTEQ.  
11  * Lombok genera getters, setters, constructores y toString  
12  * automaticamente.  
13  */  
14 @Entity
```

```
13 @Table(name = "estudiantes",
14         indexes = @Index(name = "idx_cedula", columnList = "
15             cedula"))
16 @Data
17 @NoArgsConstructor
18 @AllArgsConstructor
19 @Builder
20 public class Estudiante {
21     @Id
22     @GeneratedValue(strategy = GenerationType.IDENTITY)
23     private Long id;
24
25     @NotBlank(message = "La cedula es obligatoria")
26     @Pattern(regexp = "\\d{10}", message = "La cedula debe
27         tener 10 digitos")
28     @Column(nullable = false, unique = true, length = 10)
29     private String cedula;
30
31     @NotBlank(message = "El nombre es obligatorio")
32     @Size(min = 2, max = 100, message = "Nombre entre 2 y 100
33         caracteres")
34     @Column(nullable = false, length = 100)
35     private String nombre;
36
37     @NotBlank(message = "El apellido es obligatorio")
38     @Column(nullable = false, length = 100)
39     private String apellido;
40
41     @Email(message = "Correo electronico invalido")
42     @Column(unique = true, length = 200)
43     private String email;
44
45     @Min(1) @Max(10)
46     @Column(nullable = false)
47     private Integer semestre;
48
49     @Column(name = "creado_en", updatable = false)
50     private LocalDateTime creadoEn;
51
52     @Column(name = "actualizado_en")
```

```
51     private LocalDateTime actualizadoEn;
52
53     @PrePersist
54     protected void onCreate() {
55         creadoEn = LocalDateTime.now();
56         actualizadoEn = LocalDateTime.now();
57     }
58
59     @PreUpdate
60     protected void onUpdate() {
61         actualizadoEn = LocalDateTime.now();
62     }
63 }
```

Listing 5.1: Estudiante.java – Entidad JPA

```
1 package ec.edu.uteq.distribuidas.repository;
2
3 import ec.edu.uteq.distribuidas.model.Estudiante;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.stereotype.Repository;
7 import java.util.List;
8 import java.util.Optional;
9
10 /**
11  * Repositorio Spring Data JPA para Estudiante.
12  * Spring genera automaticamente la implementacion de los
13  * metodos.
14  */
15 @Repository
16 public interface EstudianteRepository
17     extends JpaRepository<Estudiante, Long> {
18
19     Optional<Estudiante> findByCedula(String cedula);
20
21     boolean existsByCedula(String cedula);
22
23     List<Estudiante> findBySemestre(Integer semestre);
24
25     List<Estudiante>
```

```
        findByNombreContainingIgnoreCaseOrApellidoContainingIgnoreCase
        (
25         String nombre, String apellido);
26
27         @Query("SELECT e FROM Estudiante e WHERE e.semestre = :
                semestre " +
28                 "ORDER BY e.apellido ASC, e.nombre ASC")
29         List<Estudiante> buscarPorSemestreOrdenado(Integer semestre
                );
30
31         long countBySemestre(Integer semestre);
32     }
```

Listing 5.2: EstudianteRepository.java – Repositorio Spring Data JPA

```
1 package ec.edu.uteq.distribuidas.service;
2
3 import ec.edu.uteq.distribuidas.dto.*;
4 import ec.edu.uteq.distribuidas.exception.*;
5 import ec.edu.uteq.distribuidas.model.Estudiante;
6 import ec.edu.uteq.distribuidas.repository.EstudianteRepository
7     ;
8 import lombok.RequiredArgsConstructor;
9 import lombok.extern.slf4j.Slf4j;
10 import org.springframework.data.domain.*;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional
13     ;
14 import java.util.List;
15
16 /**
17  * Servicio de negocio para la gestion de estudiantes.
18  * Contiene toda la logica: validaciones, transformaciones y
19  * coordinacion.
20  */
21 @Service
22 @RequiredArgsConstructor
23 @Slf4j
24 @Transactional(readOnly = true)
25 public class EstudianteService {
```

```
24     private final EstudianteRepository repo;
25
26     /** Lista estudiantes con paginacion y ordenamiento. */
27     public Page<EstudianteResponse> listar(int pagina, int
28         tamano,
29
30         String ordenarPor)
31     {
32         Pageable pageable = PageRequest.of(pagina, tamano,
33             Sort.by(ordenarPor).ascending());
34         return repo.findAll(pageable).map(this::toResponse);
35     }
36
37     /** Busca un estudiante por ID. Lanza excepcion si no
38         existe. */
39     public EstudianteResponse buscarPorId(Long id) {
40         return repo.findById(id)
41             .map(this::toResponse)
42             .orElseThrow(() -> new RecursoNoEncontradoException
43                 (
44                     "Estudiante", "id", id));
45     }
46
47     /** Crea un nuevo estudiante validando unicidad de cedula y
48         email. */
49     @Transactional
50     public EstudianteResponse crear(EstudianteRequest request)
51     {
52         log.info("Creando estudiante con cedula={}", request.
53             cedula());
54
55         if (repo.existsByCedula(request.cedula())) {
56             throw new ConflictoException("cedula",
57                 "La cedula " + request.cedula() + " ya esta
58                 registrada");
59         }
60
61         Estudiante nuevo = Estudiante.builder()
62             .cedula(request.cedula())
63             .nombre(request.nombre().trim())
64             .apellido(request.apellido().trim())
65             .email(request.email())
```

```
57         .semestre(request.semestre())
58         .build();
59
60     Estudiante guardado = repo.save(nuevo);
61     log.info("Estudiante creado con id={}", guardado.getId
62         ());
63     return toResponse(guardado);
64 }
65
66 /** Actualiza los datos de un estudiante. */
67 @Transactional
68 public EstudianteResponse actualizar(Long id,
69     EstudianteRequest request) {
70     Estudiante existente = repo.findById(id)
71         .orElseThrow(() -> new RecursoNoEncontradoException
72             (
73                 "Estudiante", "id", id));
74
75     existente.setNombre(request.nombre().trim());
76     existente.setApellido(request.apellido().trim());
77     existente.setEmail(request.email());
78     existente.setSemestre(request.semestre());
79
80     return toResponse(repo.save(existente));
81 }
82
83 /** Elimina un estudiante. Lanza excepcion si no existe. */
84 @Transactional
85 public void eliminar(Long id) {
86     if (!repo.existsById(id)) {
87         throw new RecursoNoEncontradoException("Estudiante"
88             , "id", id);
89     }
90     repo.deleteById(id);
91     log.info("Estudiante id={} eliminado", id);
92 }
93
94 /** Busca por semestre con orden. */
95 public List<EstudianteResponse> buscarPorSemestre(Integer
96     semestre) {
97     return repo.buscarPorSemestreOrdenado(semestre)
```



```
93         .stream().map(this::toResponse).toList();
94     }
95
96     /** Convierte entidad a DTO de respuesta. */
97     private EstudianteResponse toResponse(Estudiente e) {
98         return new EstudianteResponse(
99             e.getId(), e.getCedula(), e.getNombre(), e.
100                 getApellido(),
101             e.getEmail(), e.getSemestre(), e.getCreadoEn()
102         );
103     }
104 }
```

Listing 5.3: EstudianteService.java – Logica de negocio

```
1 package ec.edu.uteq.distribuidas.controller;
2
3 import ec.edu.uteq.distribuidas.dto.*;
4 import ec.edu.uteq.distribuidas.service.EstudianteService;
5 import jakarta.validation.Valid;
6 import lombok.RequiredArgsConstructor;
7 import org.springframework.data.domain.Page;
8 import org.springframework.http.*;
9 import org.springframework.web.bind.annotation.*;
10 import org.springframework.web.servlet.support.
    ServletUriComponentsBuilder;
11 import java.net.URI;
12 import java.util.List;
13
14 /**
15  * Controlador REST para el recurso /api/v1/estudiantes.
16  * Sigue las convenciones REST: verbos HTTP, codigos de estado
17    y HATEOAS basico.
18  */
19 @RestController
20 @RequestMapping("/api/v1/estudiantes")
21 @RequiredArgsConstructor
22 public class EstudianteController {
23
24     private final EstudianteService service;
```

```
25 // GET /api/v1/estudiantes?pagina=0&tamano=20&ordenarPor=
    apellido
26 @GetMapping
27 public ResponseEntity<Page<EstudianteResponse>> listar(
28     @RequestParam(defaultValue = "0") int pagina,
29     @RequestParam(defaultValue = "20") int tamano,
30     @RequestParam(defaultValue = "apellido") String
        ordenarPor) {
31     return ResponseEntity.ok(service.listar(pagina, tamano,
        ordenarPor));
32 }
33
34 // GET /api/v1/estudiantes/{id}
35 @GetMapping("/{id}")
36 public ResponseEntity<EstudianteResponse> buscarPorId(
37     @PathVariable Long id) {
38     return ResponseEntity.ok(service.buscarPorId(id));
39 }
40
41 // GET /api/v1/estudiantes/semestre/{semestre}
42 @GetMapping("/semestre/{semestre}")
43 public ResponseEntity<List<EstudianteResponse>> porSemestre
    (
44     @PathVariable Integer semestre) {
45     return ResponseEntity.ok(service.buscarPorSemestre(
        semestre));
46 }
47
48 // POST /api/v1/estudiantes
49 @PostMapping
50 public ResponseEntity<EstudianteResponse> crear(
51     @Valid @RequestBody EstudianteRequest request) {
52     EstudianteResponse creado = service.crear(request);
53
54     // Construir URI del recurso creado (cabecera Location)
55     URI location = ServletUriComponentsBuilder
56         .fromCurrentRequest()
57         .path("/{id}")
58         .buildAndExpand(creado.id())
59         .toUri();
```

```
60         return ResponseEntity.created(location).body(creado);
61     }
62
63     // PUT /api/v1/estudiantes/{id}
64     @PutMapping("/{id}")
65     public ResponseEntity<EstudianteResponse> actualizar(
66         @PathVariable Long id,
67         @Valid @RequestBody EstudianteRequest request) {
68         return ResponseEntity.ok(service.actualizar(id, request
69             ));
70     }
71
72     // DELETE /api/v1/estudiantes/{id}
73     @DeleteMapping("/{id}")
74     public ResponseEntity<Void> eliminar(@PathVariable Long id)
75     {
76         service.eliminar(id);
77         return ResponseEntity.noContent().build();
78     }
79 }
```

Listing 5.4: EstudianteController.java – Controlador REST

```
1 package ec.edu.uteq.distribuidas.dto;
2
3 import jakarta.validation.constraints.*;
4 import java.time.LocalDateTime;
5
6 // Request DTO (entrada)
7 public record EstudianteRequest(
8     @NotBlank @Pattern(regexp = "\\d{10}")
9     String cedula,
10
11     @NotBlank @Size(min = 2, max = 100)
12     String nombre,
13
14     @NotBlank @Size(min = 2, max = 100)
15     String apellido,
16
17     @Email
18     String email,
```

```
19
20     @NotNull @Min(1) @Max(10)
21     Integer semestre
22 ) {}
23
24 // Response DTO (salida)
25 // En archivo separado: EstudianteResponse.java
26 // public record EstudianteResponse(
27 //     Long id, String cedula, String nombre, String apellido,
28 //     String email, Integer semestre, LocalDateTime creadoEn
29 // ) {}
```

Listing 5.5: DTOs con Java Records (EstudianteRequest.java y EstudianteResponse.java)

Guía Práctica IDE – IntelliJ IDEA – Ejecutar y probar el microservicio

Paso 1: Ejecutar la clase principal `SvcEstudiantesApplication` (clic derecho → *Run*). Debe aparecer el banner de Spring Boot y el mensaje:

```
Started SvcEstudiantesApplication in 2.341 seconds (JVM
0.543s)
```

Paso 2: Abrir el navegador o Postman en `http://localhost:8080/api/v1/estudiantes`. Se debe obtener una página vacía de resultados:

```
{"content": [], "pageable": {...}, "totalElements": 0, ...}
```

Paso 3: Crear un estudiante con Postman:

```
POST http://localhost:8080/api/v1/estudiantes
Content-Type: application/json

{
  "cedula": "1234567890",
  "nombre": "Ana Maria",
  "apellido": "Guerrero Torres",
  "email": "ana.guerrero@uteq.edu.ec",
  "semestre": 7
}
```

Respuesta esperada: HTTP 201 Created con cabecera Location.

Paso 4: Verificar la consola H2 en <http://localhost:8080/h2-console>. JDBC URL: `jdbc:h2:mem:testdb`. Ejecutar:

```
SELECT * FROM ESTUDIANTES;
```

Paso 5: Probar validaciones: intentar crear un estudiante con cédula repetida. Debe retornar HTTP 409 Conflict.

Paso 6: Verificar el Actuador: <http://localhost:8080/actuator/health>. Debe retornar `"status": "UP"`.

Ejercicio Propuesto: Agregar Swagger/OpenAPI al microservicio

1. Agregar la dependencia `springdoc-openapi-starter-webmvc-ui:2.5.0` al `pom.xml`.
2. Acceder a <http://localhost:8080/swagger-ui.html>.
3. Agregar anotaciones `@Operation` y `@ApiResponse` a cada endpoint del controlador para documentar los casos de éxito y error.
4. Exportar la especificación OpenAPI en formato JSON desde <http://localhost:8080/v3/api-docs>.

Actividad Práctica: Taller: Diseño de Arquitectura de Microservicios del PFC (FORM-TL-02)

Objetivo: Diseñar la arquitectura completa del Proyecto Final de Curso.

Instrucciones (en grupos de 3-4):

1. Definir el dominio de negocio del PFC usando Domain-Driven Design (DDD). Identificar bounded contexts.
2. Diseñar al menos 4 microservicios independientes con base de datos propia.
3. Crear el diagrama de arquitectura con: API Gateway, microservicios, bus de mensajes y BDs.
4. Clasificar las comunicaciones: síncronas (REST) vs. asíncronas (Kafka).
5. Identificar 3 patrones de microservicios a aplicar (Saga, CQRS, Circuit Breaker...).
6. Presentar en 10 minutos.